

Received September 18, 2020, accepted November 2, 2020, date of publication November 10, 2020, date of current version November 23, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.3037236

MeshGraphNet: An Effective 3D Polygon Mesh Recognition With Topology Reconstruction

AN PING SONG¹, XIN YI DI¹, XIAO KANG XU¹, AND ZI HENG SONG²

¹School of Computer Science and Technology, Shanghai University, Shanghai 200444, China

²College of Engineering, University of Illinois at Urbana Champaign, Champaign, IL 61820, USA

Corresponding author: An Ping Song (apsong@shu.edu.cn)

ABSTRACT Three-dimensional polygon mesh recognition has a significant impact on current computer graphics. However, its application to some real-life fields, such as unmanned driving and medical image processing, has been restricted due to the lack of inner-interactivity, shift-invariance, and numerical uncertainty of mesh surfaces. In this paper, an interconnected topological dual graph that extracts adjacent information from each triangular face of a polygon mesh is constructed, in order to address the above issues. On the basis of the algebraic topological graph, we propose a mesh graph neural network, called MeshGraphNet, to effectively extract features from mesh data. In this concept, the graph node-unit and correlation between every two dual graph vertexes are defined, the concept of aggregating features extracted from geodesically adjacent nodes is introduced, and a graph neural network with available and effective blocks is proposed. With these methods, MeshGraphNet performs well in 3D shape representation by avoiding the lack of inner-interactivity, shift-invariance, and the numerical uncertainty problems of mesh data. We conduct extensive 3D shape classification experiments and provide visualizations of the features extracted from the fully connected layers. The results demonstrate that our method performs better than state-of-the-art methods and improves the recognition accuracy by 4–4.5%.

INDEX TERMS Algebraic topology reconstruction, effectively feature extracting method, graph convolutional method, mesh recognition.

I. INTRODUCTION

Three-dimensional polygon meshes are one of the most crucial forms of 3D shape representation in the field of computer graphics and computer vision, which can be used for 3D shape recognition and classification. Due to the lack of an appropriate generic shape representation for 3D data in computer systems, meshes have remained an unsolved puzzle in 3D shape representation over the past two decades. Chang *et al.* [1] and Wu *et al.* [2] have recently provided significant contributions to 3D shape recognition and classification tasks. For 3D shape representation, there exist a number of typical representations, such as volumetric grids, multi-views, point clouds, meshes. As convolutional neural networks (CNNs) have achieved great success in the field of image recognition, many researchers have attempted to apply CNNs to 3D representations. Wu *et al.* [2] applied a CNN to a volumetric grid to extract voxel features for

classification, and have achieved 77% recognition accuracy. Riegler *et al.* [3] introduced the sparse 3D data storage method, which reduced space consumption and computational resources by employing octree. Su *et al.* [4] combined the features extracted by a CNN from multiple pictures in order to improve the classification accuracy. Qi *et al.* [5] proposed a neural network to solve permutation-invariance and the shift-invariance, which can be applied to analyze point cloud data. Qi *et al.* [6] proposed multi-scale grouping (MSG) and multi-resolution grouping (MRG) to enhance the relevance among sparse data. From the idea proposed by Qi *et al.* [5], a large number of variants have emerged, such as Frustum [7], Pu-net [8], and so on. Polygon mesh data are a collection of vertices, edges, and faces, which are used to fit the shape of a polyhedral object. Kazhdan *et al.* [9], used the Spherical Harmonic descriptor (SPH) to analyze hand-crafted features directly. Meanwhile, Hubeli *et al.* [10] introduced the Second-Order Difference (SOD) and Extended Second-Order Difference (ESOD) to evaluate the angle between normal vectors for classification. In recent years,

The associate editor coordinating the review of this manuscript and approving it for publication was Donghyun Kim.

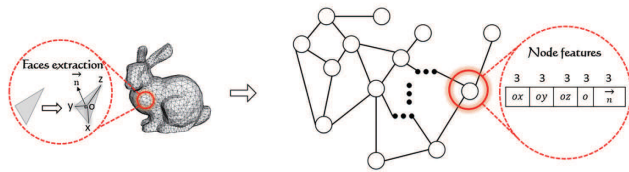


FIGURE 1. The white circles in the figure represent triangular surfaces, and the black lines, which denote dual edges, represent the correlation of adjacent triangular surfaces. Each node contains 15-dimensional features, in which ox occupies 3-dimensional spatial co-ordinates, also occupied by oy , oz , o , and n .

much attention has been paid to meshes, as they contain much more texture information and provide more effective shape representations than other 3D representations. For example, Gkioxari [11] cubified the 3D voxel occupancy probability in a triangular mesh model using the topology predicted from the mesh branch. Wen [12] and Pan [13], [14] have made significant contributions to the field of 3D reconstruction. However, the 3D data format faces the problem that the correlation between every two candidates of a particular 3D type is unconnected, especially in point clouds and polygon meshes. For example, PointCNN [33] learns an x-transformation autoencoder via neural network to permute the input order of point cloud data, which is then used to solve the permutation-invariance of point cloud data, further applying a grid-based CNN to extract features to enhance the inner connection between two points. Similarly, DGCNN [34] defines Euclidean neighborhoods and uses edge convolution to enhance data correlation; an idea similar to that of KPConv [35].

In this paper, inspired by Moas *et al.* [14] and Felzenszwalb *et al.* [15] and GumholdS *et al.* [16], we propose the algebraic topology of dual graph $G = \{N, E\}$ (see Figure 1), where N are the nodes set that contains features extracted from mesh data, and E are the dual edges set that represent the relationships between every two mesh surfaces. Importantly, we extract the principal features of the mesh data to define the topological space $N = \{ox, oy, oz, o, n\}$, where ox , oy , and oz are vectors from a central point to the three unordered vertices, o is the geometric central point calculated from three vertices of the triangular face, and n is the spatial normal vector of the triangular face, which represents the spatial information. In this way, we split the geometric and spatial features of the topological space into ox , oy , oz , o and n , respectively. Furthermore, similar to Hubeli *et al.* [10], we redefine the correlation between every two faces, in order to solve the problem of the lack of inner-interactivity of the topology. On the basis of the dual graph, we design a graph neural network which applies the designed graph convolution method to extract features and combines the separated features together for classification. By using this network, we are able to address the challenges posed by the lack of inner-interactivity, shift-invariance, and numerical uncertainty of mesh surfaces of mesh data.

By applying our MeshGraphNet method to 3D shape classification on ModelNet40 [1], the results show that our

MeshGraphNet method outperforms other existing 3D representation methods. Below are the following advantages of the new proposed MeshGraphNet method:

- The novel data structure of mesh data (called a mesh graph) can directly handle a mesh data set and redefine the geodesic neighborhoods of each surface.
- The new graph neural network (<https://github.com/JsBlueCat/MeshGraph>) is able to enhance the accuracy of 3D shape classification and the new graph convolution method can produce more correlated mesh data.
- According to the results of our extensive experiments, the new graphical method performs well in 3D shape classification and is able to achieve a classification accuracy of up to 94.2%.

II. RELATIVE WORK

The deep learning methods which are used to solve the above problems, considering different 3D data formats, can be classified into the following four categories:

- 1) Volumetric Grids. Three-dimensional voxel data are transformed into a volumetric grid by imitating the 2D pixel format, such that all operations that are conducted in 2D can be extended to 3D, thus guaranteeing the inner-correlation of every two candidates. VoxNet [17] applies a CNN to process the voxels of a volumetric occupancy grid for 3D object classification. Wang *et al.* [18] used a spatial octree to represent a 3D model and performed 3D CNN operations with normal vectors extracted from the model surface, which is sampled at the leaf nodes. Ren *et al.* [19] applied generative adversarial networks to voxel data.
- 2) Multi-Views. Multiple faces of the 2D domain are mapped through the current image's auto-encoder and decoder network into 3D space. Su *et al.* [4] used 2D images to classify 3D shapes. Furthermore, on the basis of multi-views, a loop mechanism is added to combine an RNN and CNN, and multiple attempts are made to extract features and map them into 3D space, as proposed by Jiang *et al.* [20].
- 3) Point Clouds. A set of data points in space is used to represent the shape of an object. PointNet [5] uses a symmetry function to solve the permutation invariance of the data points, as well as the invariance under transformations, while ignoring the interactions among points. PointNet++ [6] abstracts local features through a multiscale learning algorithm to solve the same problem. Furthermore, Frustum [7] tapers the 3D proposal area to solve semantic segmentation tasks. PointCNN [33] introduces an x-transform function to enhance the correlation within each point, such that a convolutional function can be conducted on it. DGCNN [34] defines Euclidean neighborhoods and uses edge convolution to enhance data correlation.
- 4) Meshes. Various discrete approximations for 3D shapes have been suggested and utilized to represent shapes in

an array of applications. Among them, the polygonal mesh representation, (or mesh, for short) approximates surfaces using a set of 2D polygons in 3D space, as discussed in Botsch *et al.* [21]. MeshCNN [22] introduces an edge collapse operation to work directly on a mesh and solves the complexity to reduce space occupation. MeshNet [23] uses a neural network that extracts features from the surface, which is used for 3D classification. Gkioxari [11] reconstructed the topology to cubify the 3D voxel occupancy probability to a triangular mesh and reconstructed the image as a 3D mesh.

In contrast to deep learning methods, the approach proposed by Felzenszwalb [15] classifies images by reconstructing graphs that extract pixel intensities. Cignoni [24] proposed a number of mesh simplification operations, in order to solve the complexity of the mesh. Hubeli [10] proposed a special method which extracts normal vectors. Similarly, Kalvin [25] applied the main features of normal vectors to classify boundaries. Kipf [26] proposed a graph convolutional method for classification by using neural networks. Hamilton [27] applied the method in Kipf [26] for large graph classification. Xu [28] proposed an efficient graph convolutional network for graph classification. Fey [29] proposed a graph convolutional framework.

Since triangular meshes usually carry a lot of spatial geometric information which differs from other three-dimensional data. In this article, we propose an algebraic topology of spatial geometry, and use graphs to represent the spatial adjacency of triangular meshes. This method combines some of the early ideas([10], [25]) for the extraction of triangular mesh features with the perception function of deep learning([5], [6], [22], [23]). Generally speaking, we propose auto-encoder function which is applied to map the effective features to the latency dimension space (whose size is same as the size of original dimension) to achieve the permutation-invariance and shift-irrelevance features, and some graph convolution methods are defined to process the encode features to aggregate spatial connection information to the final classification feature.

III. DESIGN OF MeshGraphNet

In this section, we discuss how MeshGraphNet is designed. By considering the strengths and weaknesses of existing approaches such as MeshCNN [22], MeshNet [23], Felzenszwalb [15], SOD and ESOD [10], and Kalvin [25], we reconstruct a algebraic topological dual graph that contains adjacency information and extracts the normal vectors of each surface and spatial co-ordinates of vertices as spatial normal line information and spatial plane information, respectively (see Figure 1). Furthermore, we propose symmetrical graph convolutional functions that extract features from shift-invariance mesh data using the rules of k -nearest geodesic neighborhoods, which we call the Shape Approximation Module (SAM). In general, we propose an effective and available neural network (see Figure 2) that aggregates the features extracted from the dual graph for 3D tasks.

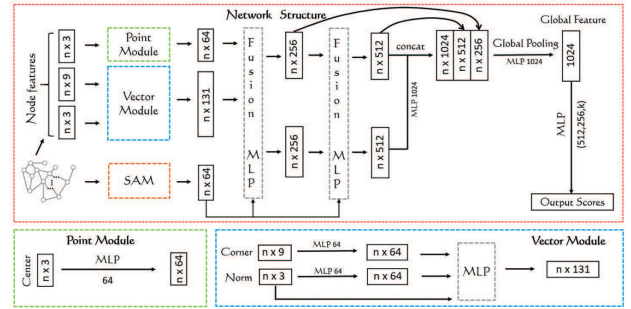


FIGURE 2. The node features are split into the Point Module and Vector Module. The normal vector is extracted from the designed dual graph using the designed graph convolutional Spatial Approximation Module(SAM) 4, passing by the multi-layer perceptron(MLP) to transform to a latent permutation. All features extracted by the above modules are combined together for further recognition tasks. First, the input of the data will be a.obj file or other mesh data. Second, the data will be reconstructed into an algebraic topological dual graph. Last, all modules proposed above will conduct on the graph to get the classification features which will be predicted by a softmax layer to get k classification scores, and the k is restricted to the categories numbers, such as, 40 of ModelNet40.

A. ALGEBRAIC TOPOLOGICAL DUAL GRAPH RECONSTRUCTION

As mesh data contain many complex features which might not be favorable for computational tasks (e.g., texture and lighting information), we extract the normal vector of each surface and the spatial co-ordinates of the vertices as spatial normal line information and spatial plane information, respectively. As a result, we define topological spaces $N = \{ox, oy, oz, o, n\}$, where o is the geometric central point calculated from three vertices of the triangular face; ox , oy , and oz are the vectors from the central point to the three unordered vertices, respectively; and n is the normal vector of the triangular face that represents the spatial normal line information. In addition to node reconstruction, we define dual edges set E as the correlation between every two nodes, which means that the two vertices are related when there exists some overlap in the associated vertices or edges, which is also known as separated dual edge. For example, two nodes v_1 and v_2 are geodesic neighborhoods where they share the same dual edge. As such, we define a algebraic topological dual graph (see Figure 1) in which the complex mesh data are transformed into an undirected dual graph, in order to connect all surfaces in a graph data structure format, thus solving the problem of the lack of inner-interactivity of mesh data.

B. MODULE DESIGN FOR MeshGraphNet

In this section, we introduce the designed modules, which are applied to address the lack of inner-interactivity, permutation-invariance, and the numerical uncertainty of mesh surfaces of mesh data. The fusion multi-layer perceptron (fusion MLP Figure 2) is introduced to solve the permutation-invariance of input mesh data, which using symmetrical activator of neurons to transfer features into a latent permutation. With the designed symmetrical activator function, three corner vectors are transformed into blended features, regardless of the order of input. Then, the features can be split into two

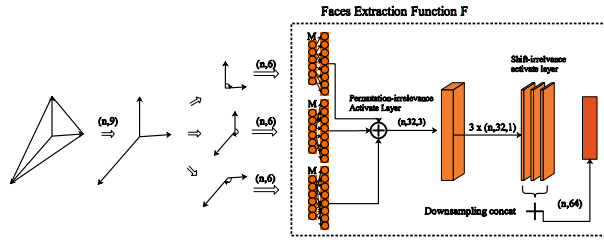


FIGURE 3. The corner vectors of the triangular mesh will be divided into three parts. Any two corner vectors will be extracted into a high-dimensional feature space by the neuron perceptron M with shared weights, and a disorderly arranged superimposed activation layer (by Equation (1)). The auto-encoder up-samples the feature information to a high-dimensional space and reduces the dimensionality to a low-dimensional space to ensure its rotation disorder.

topological space set that are spatial plane factors and spatial line factors.

For spatial plane factors, central points are regarded as the key representation for the co-ordinate system scale of the object being recognized, as they contain the spatial co-ordinates of vertices, which can model 3D objects in the real world. Similar to the scale factors in the field of image recognition, these central points are projected onto a global co-ordinate system, which is defined by using the maximum co-ordinate value as the reference co-ordinate system. In addition to the central points, three corner vectors can also be taken to calculate the surface information of the 3D object; for example, the vector (ox, oy, oz) is divided into (ox, oy) , (oy, oz) , (ox, oz) and extracted by the symmetrical auto-encoder, which is activated by a sum layer, which can be shown in Figure 3. First, given the three vectors ox, oy, oz , each of two vectors will be handled by a shared weight neuron layer. Second, features will be encoded to a latent permutation and then pass to a shift-invariance activation neuron layer P . Thus surfaces auto-encoder F has the ability to get a shift-invariance and permutation-invariance features.

$$F(ox, oy, oz) = P(M(ox, oy) + M(oy, oz) + M(oz, ox)) \quad (1)$$

However, these features face a grate problem that surfaces of the topological space set N are lack of strong correlation, due to the isolation of algebraic topological points(N). Therefore, the efficient use of dual edges set E is very necessary for mesh processing, which is the reason why we introduce a more innovative dual edge set E and use graph convolution module to get a encoded features. For spatial line factors, normal vectors are treated as the calculator factor of every two surfaces of the mesh data. As one direction can uniquely determine a plane, such as its curvature, angle, and so on, many papers, such as that of Hubeli [10], have proved that the inner-relationship between every two normal vectors greatly affects object recognition. Inspired by Hubeli's idea, a correlation convolutional function is designed in this paper, in order to extract the local features and enhance the neurons of the classification layer.

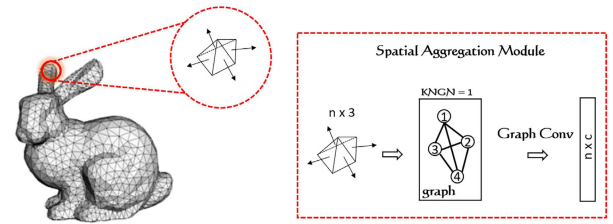


FIGURE 4. The spatial aggregation module is designed to transform the spatial normal vectors with neighbor information aggregated into the topology, which follows the k -nearest geodesic neighbors (KNGN) algorithm. At the same time, it can be extracted by our designed shift-invariance convolutional function.

In addition to the split features, a new spatial aggregation module (SAM) is also designed to aggregate the spatial normal vector with the information of its geodesic neighbors by applying the graph convolution method to extract the normal vectors and their neighbor information from the designed algebraic topological dual graph. The spatial aggregation module is shown in Figure 4, which aggregates the neighboring normal vectors through a shift-invariant convolutional function and extracts the relationship between the two normal vectors through an interactional graph convolutional function. The spatial aggregation module is not only composed of a 3×64 MLP function and a 64×64 shift-invariance function, but is also aggregated by the designed graph convolutional method, with the average activation function parsing the average of adjacent neighbors through the neuron layers to ensure the shift-invariance of the mesh data. As 3D graphic models contain a lot of local features with distinctive spatial factors, the spatial aggregation module is able to integrate the local features into compound global features. This leads to a phenomenon whereby the spatial factors perform better than other existing methods. Meanwhile, it is able to recognize the relativity of the adjacent normal vectors in the field of 3D model classification. Compared with existing methods, which use spatial normal vectors for computation factors, the spatial aggregation module has superior performance in shift-invariance and flexibility, through the use of the effective deep learning neuron layers and interactive graph convolution method.

To combine the split features, we propose a network structure (see Figure 2) that includes a point module, a vector module, the spatial aggregation module (SAM), the fusion MLP module(see Figure 5), and a global max pooling function. To solve the shift-invariance of the input mesh data, 64×64 MLP functions are used to transform the data into a shift-invariant spatial co-ordinate system. The point module, vector module, and spatial aggregation module are used to calculate the split shape factors, the split spatial factors, and the correlation between every two normal vectors, respectively. The extracted features are transformed by a shift-invariant auto-encoder and integrated by a fusion auto-encoder, which combines the fixed features and imitates the feature pyramid functions in the field of images. Finally, a global max pooling function is introduced to pool the

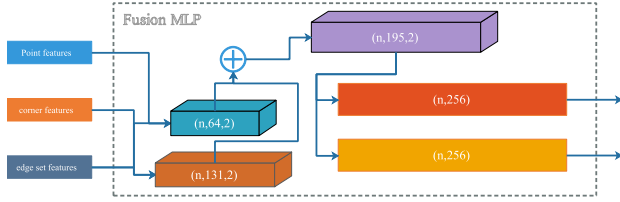


FIGURE 5. A fusion MLP module consumes two different dimensional features, and use the pyramid stacking function to map the features of different dimensions to higher dimensions, that is, the dimensional increase operation, and then use different convolution modules for dimensionality reduction and connection, and finally output local features and global features.

mixed features into 1024-dimensional classification features. By introducing pyramid fusion functions which combine multi-level global and local features, the features of fully connected layers are enhanced, leading to the model's superiority in 3D recognition.

C. DESIGN OF GRAPH CONVOLUTIONAL FUNCTION

From previous studies, such as that of Hubeli [10], it is well-known that the interaction between two spatial normal vectors plays an important role in the computational factors of mesh data classification. However, in recent decades, research has rarely adopted these early methods due to the popularity of deep learning. In our research, we combine the spatial normal vector method with a deep learning method by introducing a specific graph convolutional function. We employ the proposed mini-batch strategy by using a sparse matrix which combines the features from the graph convolutional function and apply it to solve the irregularity of the input mesh data. In order to extract the adjacency interaction between two spatial normal vectors, all edges in the designed topology are defined to be correlated between every two nodes, which are defined to be surfaces of the mesh data. Consequently, the structure of the topology is similar to that of the image data. For example, the pixels correspond to the nodes in the topology and the grid structure between two pixels is similar to the dual edges between two nodes, which is particularly suitable for convolutional functions. Therefore, an k-nearest geodesic neighbors algorithm (similar to the k-nearest neighbors (knn) clustering algorithm) is also defined to extract adjacent surface information from the correlated nodes. In our experiments, k was set to 1, which indicates that each surface interacted with its surrounding surfaces. Since generalizing the convolution operator to irregular domains is typically expressed as a neighborhood aggregation or message passing scheme, for example, in the image field, the convolution operation is defined as a computation that a pixel will generate a convolution relationship with its surrounding pixels, that is, the dot product operation, and the calculation is affected by the receptive field as the scale of convolution. Similar to the image domain, in the topological graph domain, the convolution operation can be realized by defining the relationship between a node pair and the surrounding area and defining the scope. Thus A new

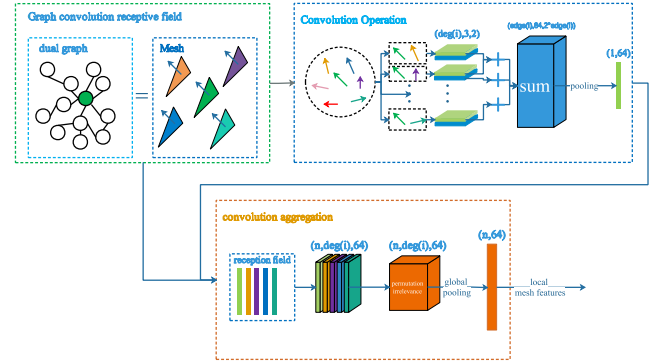


FIGURE 6. For the GEMM operation designed for graph convolution, all nodes in the algebraic dual graph will have a relationship with its neighboring nodes, and the convolution operation will act on the space normal vectors of any two dual nodes. First, using the dihedral angle calculation designed in Equation (2) to obtain features with adjacent angle relationships. Second, these functions will use the idea of hubeli [10], and use the cumulative function to aggregate as well as upsample the data to higher dimensions to ensure the permutation of the data. Third, using the pooling function to project the high-dimensional features to the low-dimensional plane to obtain the nodes and the surrounding environment. Last but not least, using the above operation on all nodes in the convolution receptive field, and design the convolution aggregation module to aggregate these low-level features, and use the disordered mapping space function to ensure disorderly arrangement, and finally output each batch of the local characteristics of the mesh block.

graph convolutional operation(see Figure 6) is designed to extract the interaction between every two normal vectors, as shown in Equation (2).

$$S(n_i, n_j) = \sum_{j \in ADJ(i)} MLP \left(\cos^{-1} \left(\frac{n_i \cdot n_j}{\|n_i\| * \|n_j\|} \right), n_i, n_j \right)$$

$$n'_i = H_\theta \left(S(n_i, n_i) + \frac{1}{\deg(n_i)} \sum_{j \in ADJ(i)} S(n_i, n_j) \right) \quad (2)$$

where H_θ represents the shift-invariant auto-encoder, applied to transform the combined features into shift-irrelevance features n' , and $S()$ is the spatial information extraction function activated by the shift-invariant neuron layer, which encodes a discrete directional curvature of each two surfaces. n_i is the i -th spatial normal vector in the one-batch topological space. The MLP auto-encoder takes in the dihedral angle of each two adjacent normal vectors and both two vectors, thus assuring the permutation-irrelevance of every dual edges set. In this paper, the angle between every two normal vectors is applied to extract the spatial information, n denotes the spatial normal vectors, $\deg()$ means the degree of node n_i which can be easily calculated by the dual graph theorem, and $ADJ(i)$ is a neighbor node set of node n_i , which is restricted by the k-nearest geodesic neighbors algorithm (KNGN).

$$GEMM(n_i, n_j)$$

$$= \sum_{j \in ADJ(i)} \sum_{j \in ADJ(i)} MLP \left(\cos^{-1} \left(\frac{n_i \cdot n_j}{\|n_i\| * \|n_j\|} \right), n_i, n_j \right)$$

$$n'_i = H_\theta \left(S(n_i, n_i) + \frac{1}{\deg(n_i)} GEMM(n_i, n_j) \right) \quad (3)$$

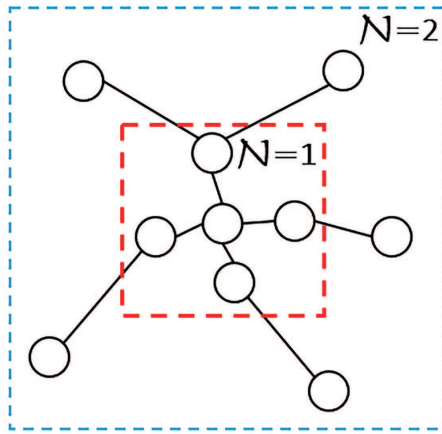


FIGURE 7. The figure shows the k-nearest geodesic neighbors algorithm, where the red rectangle indicates the receptive field of the graph convolutional function, which means that the spatial normal vector will interact with the adjacent nodes with a path equal to 1. Similarly, the blue lines represent the receptive field with a path equal to 2.

More importantly, the convolution between every nodes in topological space will contain a uncertainty number of edges, due to one surface will be separated by uncertain numerical dual edges. Also, the neighbor convolution in the Equation (2) can be simplified to Equation (3). The *GEMM* is the General Matrix Multiplication of features set of special topological space and can be realized with Fey [29]'s framework (also can be found in our repository <https://github.com/JsBlueCat/MeshGraph>).

D. K-NEAREST GEODESIC NEIGHBORS (KNGN)

Similar to NGCNN [32], the correlation defined by the geodesic method can be extracted more effectively. In order to better define the connections within the normal vectors, the receptive field of the graph convolution is affected by the k-nearest geodesic neighbors algorithm, as inspired by the ESOD method of Hubeli [10]. When the path is 1, the convolution operation is the same as SOD; when it is 2, the convolution operation is similar to ESOD. After introducing this method, the number of uncertain correlations between each spatial normal vector can strongly interact with each other.

IV. RESULTS

In this section, we first transform 3D mesh data into the designed algebraic topological dual graph, which is then applied to classification tasks. We then propose a new graph Mini-batch Strategy with mesh, simplifying the pooling function to solve the numerical uncertainty of the mesh surface problem. We conducted ablation experiments to compare our methods with existing methods, in terms of time and space consumption. Finally, we visualize the fully connected layers to reveal the details of the comparison between our methods and existing methods.

A. MESH PREPROCESSING

As a mesh may contain an uncertain number of surfaces, causing difficulties in processing the neural network, we apply a

mesh simplification operation, which is similar to the pooling function in image recognition. Applying mesh collapsing options to the uncertain number of surfaces maintains the major features of mesh data, doing little harm to the feature representation. As a result, the uncertain number of mesh surfaces will collapse to a certain number of surfaces while maintaining the uncertain number of correlations between every two surfaces; for example, when applying the mesh simplification pooling function, Mesh *A* (with N_a nodes and E_a edges) and Mesh *B* (with N_b nodes and E_b edges) will collapse to a certain number of nodes (N) while maintaining the uncertain correlations between E_a and E_b . It is well-known that E will become larger than N , as each node will be surrounded by p_i ($p_i > 1$) nodes and as an undirected graph contains $2N$ edges, leading to E being equivalent to $\sum_{i=0}^N (p_i \cdot N \cdot 2)$. These uncertain edges enhance the correlation within the graph, providing greater benefit to the convolution kernels. Besides the above operation, we also align the co-ordinate system with the mesh data, similar to adjusting the resolution of images.

B. MINI-BATCH TRAINING STRATEGY FOR THE SPARSE MATRIX OF THE MESH

After reconstructing the dual graph that contains the spatial information and correlations between each spatial feature, the proposed Mini-Batch Strategy for graphs is applied, in order to effectively use our graph model. All the mesh data surfaces are reconstructed sparsely, such that the matrices in the mini-batch are stored in the same partitioned matrix. On the basis of the same structure of all the reconstructed nodes in the graph, the mini-batch concept of multi-graph classification is introduced, as shown in Equation (4):

$$G = [N|E] = \begin{bmatrix} N_{11} & N_{12} & \cdots & N_{1j} & E_1 \\ N_{21} & N_{22} & \cdots & N_{2j} & E_2 \\ \vdots & & & \vdots & \vdots \\ N_{i1} & N_{i2} & \cdots & N_{ij} & E_i \end{bmatrix}, \quad (4)$$

where the dual algebraic graph G consist of the topological co-ordinate feature space N and dual edges set E , and N_{ij} represents the information of the partitioned matrix and N is the sparse partitioned matrix having the storage format. We convert all the topological feature space into $batch \times m \times 15$ representations, such that we can follow the mini-batch idea for backward propagation, due to the neural networks are usually trained in a batch-wise fashion. $batch$ is the size of topological feature space of all the nodes in the partitioned matrix and m is the number of mesh-simplified surfaces. The proposed method extracts 15-dimensional feature vectors and applies the above *GEMM* method with the speed of near $O(t \log(t))$.

C. EXPERIMENTS OF 3D SHAPE CLASSIFICATION

We applied the proposed MeshGraphNet to the ModelNet40 data set [2] for classification tasks and carried out an ablation experiment to illustrate whether graph

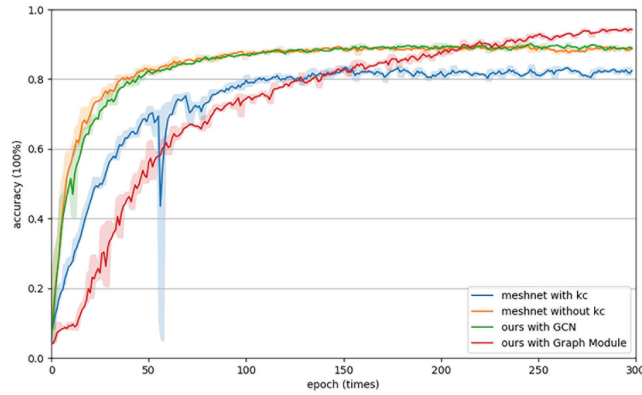


FIGURE 8. Ablation experiments were carried out on the Model40 data set for classification. Training included 300 epochs with a maximum classification accuracy of 94.3%.

convolution method applied on algebraic topology dual graph will affect the classification accuracy. The data set is composed of 12,311 mesh models with 40 categories, among which 9843 models were used for training and 2468 models were used for testing.

We apply the Adam optimizer of the gradient descent algorithm with batch size equals 64 to train our neural network. For classification tasks, we apply CrossEntropyLoss for categories classification, and we evaluate the accuracy of the top-1 classification results for the extracted features in 40 categories by using the classification layers such as Softmax layers, and the results come that the KNGN algorithm with graph convolution outperforms all current state-of-the-art correlation enhancement functions, such as Kernel Correlation (KC) [31], as shown in Figure 8. From the experimental results, it was found that KC is unfavorable for the overall experimental treatment. After adopting our graph convolutional method and taking advantage of the spatial normal vectors, the relation extraction function was able to improve the classification accuracy. Many current state-of-the-art graphic convolution functions were applied to our graphic mesh data. For example, GCN [26] achieved little improvement, as it ignores the correlations between each spatial normal vector. Finally, after taking advantage of the correlation functions such as KC and the convolution method GCN, the designed SAM convolutional method achieved the best experimental results.

In order to further illustrate the superiority of our proposed method, many comparative experiments were carried out, as presented in Table 1. The results show that our method achieved better performance than the state-of-the-art methods, although its network convergence speed was relatively slow. In summary, the use of extensive experiments proved that the graph convolutional method can achieve model optimization and higher accuracy in 3D object recognition.

D. FEATURE VISUALIZATION

To illustrate the efficiency of our method, in this section, we visualize the features of the fully connected layers.

TABLE 1. The result of classification conducted on the ModelNet40 Data set. The results shows that our model and data process are more effective.

Method	Modality	Acc
3DShapeNets [1]	volume	77.3%
voxnet [17]	volume	83%
O-cnn [18]	volume	89.9%
Mvcnn [4]	view	90.1%
mlvcnn [20]	view	94.16%
Pointnet [5]	Point cloud	77.6%
Pointnet++ [6]	Point cloud	89.2%
PointNGCNN [32]	Point cloud	92.8%
Meshnet [23]	Mesh	91%
Ours with Graph Module	Mesh	94.3% $\pm$ 0.5%

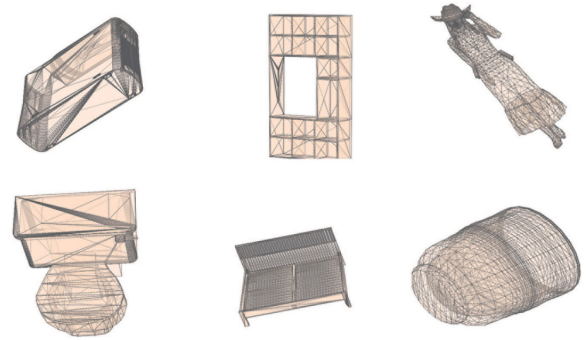


FIGURE 9. Some misclassification results detected by the network structure of MeshNet with the KC function. The top objects were always misclassified as the corresponding bottom objects, respectively. However, using the proposed graph method, the results were correct.

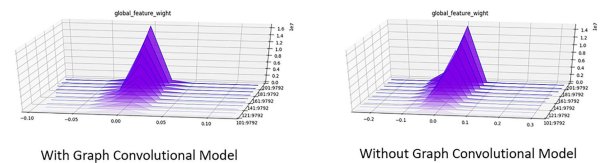


FIGURE 10. Weights of the convolutional kernel: the x-axis represents the normalized eigenvalues, whose range is from -0.5 to 0.5 , the y-axis represents the epoch and batch in the training procedure, and the z-axis represents the value of the kernel with an order of 1×10^7 .

Consequently, we find that the existing network structures, such as MeshNet [23], always misclassified some objects that are similar in shape, size, and so on, as seen in Figure 9. However, our graph convolutional method was able to classify these objects correctly. This demonstrates that our network can combine more local features with the global features. The spatial normal vectors extracted by the proposed convolutional methods had a very positive effect on the recognition process, as they contained extensive neighbor information about the surfaces.

In addition, we visualized the parameters inside the convolution kernel through the same batch, as seen in Figure 10. Specifically, a random initialization function using orthogonal initialization was applied to the model. We found that the peak of the convolution kernel features differed by 2×10^6 when our method was applied, versus not applied. It turns out that our method can be used to enhance features. Indeed, the neurons of the model that apply the graph model were

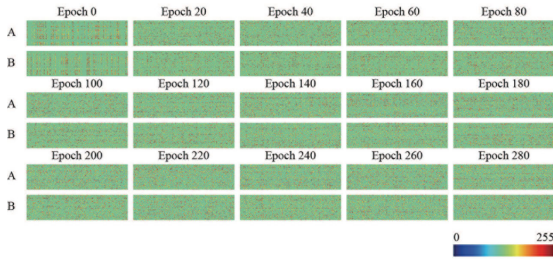


FIGURE 11. Feature visualization of fully connected layers. The picture in A depicts models with the SAM module, and B depicts them without graph modules. The picture is composed of every 20 epochs with 64×256 -dimensional features in each picture; 64 is the number of models in a batch with 64 models, and 256 is the number of features in the fully connected layer, whose dimensionality is reduced to 256 by global max pooling. Each line in the picture represents the features of the fully connected layer for classifying a model. The features range from 0 to 255, which are shown by color changes from blue to red, respectively.

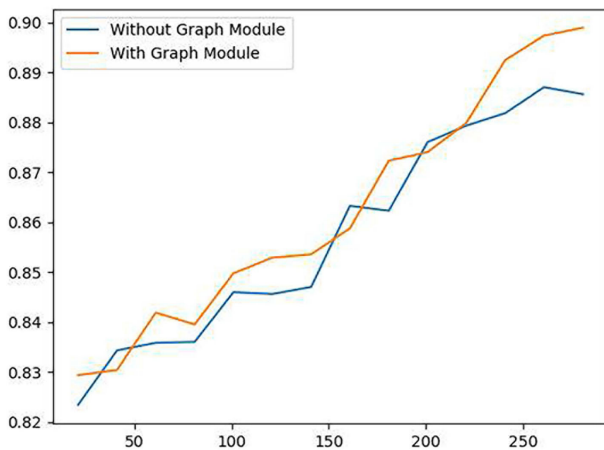


FIGURE 12. We visualized the proportion of all features ranging from $(0, 255 * 0.2] \cup [255 * 0.8, 255)$. As shown in the figure, the horizontal axis represents the number of training epochs from 0 to 300. The vertical axis represents the percentage value of all features.

more activated to capture features of the mesh data, as organized in the form of a graph. This explains why our model can improve the classification results. Moreover, we can visualize the features in the fully connected layer in a batch for the same input data. The results show that, by adding our Graph Convolutional Model, the features of the fully connected layer were significantly enhanced, as shown in Figure 11, where there was a marked increase in neurons with a particularly dark color—with darker colors indicating that the activation of the neurons is enhanced, which can obtain more correlated data. The model with the graph module performed better than the model without the graph module, as can be seen in Figure 11, in which the neurons in A are denser than the sparse ones in B, which shows that the graph module truly affected the activation of neurons in the neural network.

In general, the topological graph can maintain a theoretically infinite correlation between two spatial features simultaneously by applying a mesh simplification function to downsample the mesh data to a certain number of features using the mini-batch strategy. Indeed, if the batch equals 1,

the graph can handle an uncertain number of mesh data instances. Furthermore, the two related surfaces can be more closely calculated and the inter-relationships between the local and global features can be more closely extracted, as shown in Figures 10 and 11, respectively. This makes the classification results more efficient.

In order to further use statistical methods to illustrate the statistical changes of eigenvalues (Figure 11), we divide the extreme eigenvalues into $(0, 255 * 0.2] \cup [255 * 0.8, 255)$ intervals, and count the number of occurrences and the proportion of the total interval. ratio.

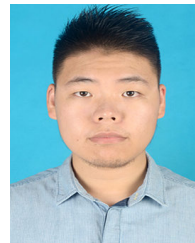
REFERENCES

- [1] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, "ShapeNet: An information-rich 3D model repository," 2015, *arXiv:1512.03012*. [Online]. Available: <http://arxiv.org/abs/1512.03012>
- [2] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, Jun. 2015, pp. 1912–1920.
- [3] G. Riegler, A. O. Ulusoy, and A. Geiger, "OctNet: Learning deep 3D representations at high resolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 3577–3586.
- [4] H. Su, S. Maji, E. Kalogerakis, and E. Learned-Miller, "Multi-view convolutional neural networks for 3D shape recognition," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Santiago, Chile, Dec. 2015, pp. 945–953.
- [5] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 652–660.
- [6] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 5099–5108.
- [7] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, "Frustum PointNets for 3D object detection from RGB-D data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 918–927.
- [8] L. Yu, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng, "PU-net: Point cloud upsampling network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 2790–2799.
- [9] M. Kazhdan, T. Funkhouser, and S. Rusinkiewicz, "Rotation invariant spherical harmonic representation of 3D shape descriptors," in *Proc. Symp. Geometry Process.*, vol. 6, 2003, pp. 156–164.
- [10] A. Hubeli and M. Gross, "Multiresolution feature extraction for unstructured meshes," in *Proc. Visualizat.*, San Diego, CA, USA, Oct. 2001, pp. 287–294.
- [11] G. Gkioxari, J. Johnson, and J. Malik, "Mesh R-CNN," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, Oct. 2019, pp. 9785–9795.
- [12] C. Wen, Y. Zhang, Z. Li, and Y. Fu, "Pixel2Mesh++: Multi-view 3D mesh generation via deformation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, Oct. 2019, pp. 1042–1051.
- [13] J. Pan, X. Han, W. Chen, J. Tang, and K. Jia, "Deep mesh reconstruction from single RGB images via topology modification networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, Oct. 2019, pp. 9964–9973.
- [14] N. Moës, J. Dolbow, and T. Belytschko, "A finite element method for crack growth without remeshing," *Int. J. Numer. Methods Eng.*, vol. 46, no. 1, pp. 131–150, Sep. 1999.
- [15] P. F. Felzenszwalb and D. P. Huttenlocher, "Efficient graph-based image segmentation," *Int. J. Comput. Vis.*, vol. 59, no. 2, pp. 167–181, Sep. 2004.
- [16] S. Gumhold, X. Wang, and R. S. MacLeod, "Feature extraction from point clouds," in *Proc. IMR*, 2001, pp. 293–305.
- [17] D. Maturana and S. Scherer, "VoxNet: A 3D convolutional neural network for real-time object recognition," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Hamburg, Germany, Oct. 2015, pp. 922–928.
- [18] P.-S. Wang, Y. Liu, Y.-X. Guo, C.-Y. Sun, and X. Tong, "O-CNN: Octree-based convolutional neural networks for 3D shape analysis," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–11, Jul. 2017.

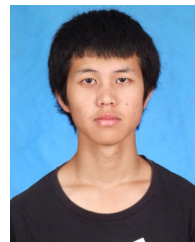
- [19] M. Ren, L. Niu, and Y. Fang, "3D-A-nets: 3D deep dense descriptor for volumetric shapes with adversarial networks," 2017, *arXiv:1711.10108*. [Online]. Available: <http://arxiv.org/abs/1711.10108>
- [20] J. Jiang, D. Bao, Z. Chen, X. Zhao, and Y. Gao, "MLVCNN: Multi-loop-view convolutional neural network for 3D shape retrieval," in *Proc. AAAI Conf. Artif. Intell.*, Honolulu, HI, USA, vol. 33, 2019, pp. 8513–8520.
- [21] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, and B. Lévy, *Polygon Mesh Processing*. Boca Raton, FL, USA: CRC Press, 2010.
- [22] R. Hanocka, A. Hertz, N. Fish, R. Giryes, S. Fleishman, and D. Cohen-Or, "MeshCNN: A network with an edge," *ACM Trans. Graph.*, vol. 38, no. 4, pp. 1–12, 2019.
- [23] Y. Feng, Y. Feng, H. You, X. Zhao, and Y. Gao, "MeshNet: Mesh neural network for 3D shape representation," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, Honolulu, HI, USA, vol. 33, 2019, pp. 8279–8286.
- [24] P. Cignoni, C. Montani, and R. Scopigno, "A comparison of mesh simplification algorithms," *Comput. Graph.*, vol. 22, no. 1, pp. 37–54, Feb. 1998.
- [25] A. D. Kalvin and R. H. Taylor, "Superfaces: Polygonal mesh simplification with bounded error," *IEEE Comput. Graph. Appl.*, vol. 16, no. 3, pp. 64–77, May 1996.
- [26] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*. [Online]. Available: <http://arxiv.org/abs/1609.02907>
- [27] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 1024–1034.
- [28] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" 2018, *arXiv:1810.00826*. [Online]. Available: <http://arxiv.org/abs/1810.00826>
- [29] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," 2019, *arXiv:1903.02428*. [Online]. Available: <http://arxiv.org/abs/1903.02428>
- [30] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jun. 2017, pp. 2117–2125.
- [31] Y. Shen, C. Feng, Y. Yang, and D. Tian, "Mining point cloud local structures by kernel correlation and graph pooling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Salt Lake City, UT, USA, Jun. 2018, pp. 4548–4557.
- [32] Q. Lu, C. Chen, W. Xie, and Y. Luo, "PointNGCNN: Deep convolutional networks on 3D point clouds with neighborhood graph filters," *Comput. Graph.*, vol. 86, pp. 42–51, Feb. 2020.
- [33] Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen, "PointCNN: Convolution on X-transformed points," in *Proc. Adv. Neural Inf. Process. Syst.*, Montréal, QC, Canada, 2018, pp. 820–830.
- [34] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, "Dynamic graph CNN for learning on point clouds," *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, 2019.
- [35] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. Guibas, "KPConv: Flexible and deformable convolution for point clouds," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, Oct. 2019, pp. 6411–6420.



AN PING SONG was born in Shanghai, China, in 1977. He received the B.S., M.S., and Ph.D. degrees in computer application from Shanghai University, China, in 2009. Since 2009, he has been a Professor with the Computer Engineering and Science Department, Shanghai University, Shanghai. He is the author of one book and more than 20 articles. His research interests include medical image processing and algorithm, bioinformatics, database application, brain-computer interface, and parallel computing.



XIN YI DI was born in Shanghai, China, in 1995. He received the B.S. degree in computer science from Shanghai University, Shanghai, China, in 2017, where he is currently pursuing the M.S. degree in computer science and technology. His research interests include image processing and algorithms, 3d object recognition, mesh processing, and machine learning.



XIAO KANG XU was born in Shanghai, China, in 1995. He received the B.S. degree in computer science from Shanghai University, Shanghai, China, in 2017, where he is currently pursuing the M.S. degree in computer science and technology. His research interests include image processing and algorithms, object recognition, brain-computer interface, and machine learning.



ZI HENG SONG was born in Shanghai, China, in 1998. He is currently pursuing the B.S. degree in computer science with Purdue University. His main research fields include image processing, data mining, and machine learning.

...